SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307
STUDENT NAME: K. Ganesh
REGISTER NUMBER: 192411170

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I K. Ganesh / 192411170 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *K. Ganesh*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

https://docs.google.com/spreadsheets/d/1YxbIZd9p1Dt05a1GKf6_1khTIAGSW9_Ytb0oXPkEgnk/edit?usp=sharing

Question 1: Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure.

The snippet is only a bare form. Several elements that give a form and its page meaning are missing:

- **`<label>`:** neither input has a label, so a screen reader cannot tell the user what "name" or "email" is for, and clicking the text does not focus the field.
- **`<fieldset>` and `<legend>`:** to group the participant details and give the group a title.
- **Page structure:** `<header>` for the portal title, `<main>` for the main content, a `<section>` with a heading (`<h2>`) around the form, and a `<footer>`.
- **Basics:** DOCTYPE, `<html lang>`, a head with charset and title, and `type="submit"` on the button.

2. Modify the structure to make it standards-compliant.

The corrected version below adds the page skeleton, semantic elements, linked labels and basic validation attributes. The method value is lowercase, which is the usual style (HTML treats it case-insensitively).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header><h1>Online Symposium Portal</h1></header>
  <main>
    <section>
      <h2>Register for the Symposium</h2>
      <form action="/register" method="post">
        <fieldset>
          <legend>Participant details</legend>
          <p><label for="name">Full name</label>
             <input type="text" id="name" name="name"
                    required autocomplete="name"></p>
          <p><label for="email">Email address</label>
             <input type="email" id="email" name="email"
                    required autocomplete="email"></p>
          <button type="submit">Register</button>
        </fieldset>
      </form>
    </section>
  </main>
  <footer><p>&copy; 2026 Symposium Committee</p></footer>
</body>
</html>
```

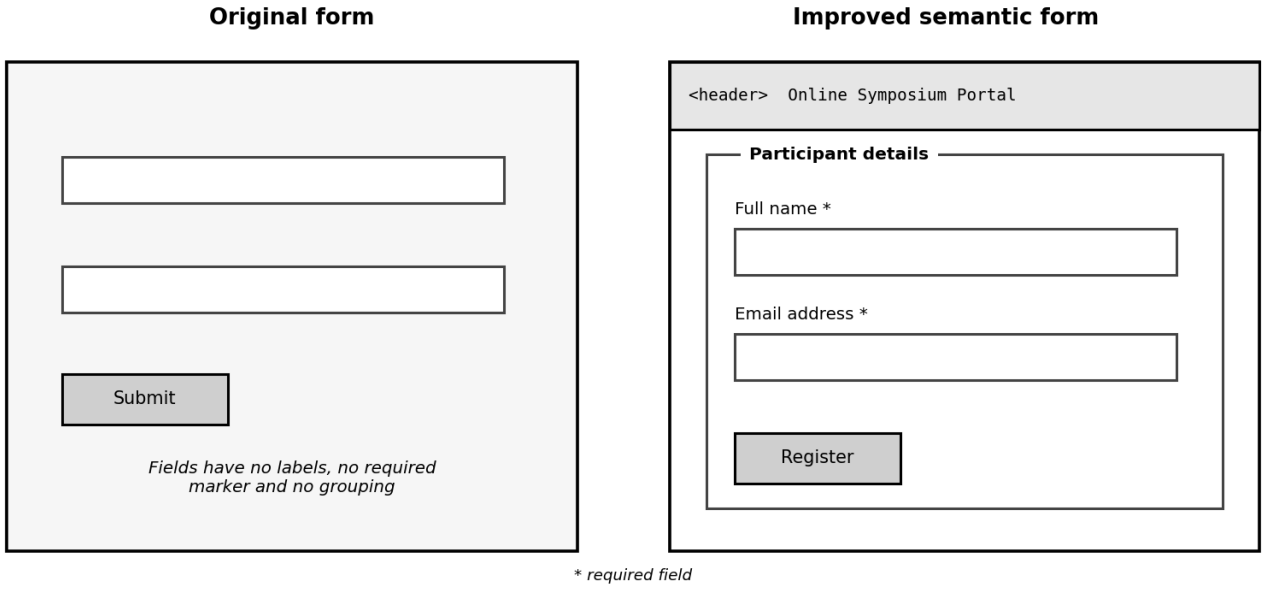


Figure 1: The original form compared with the improved semantic form

3. Interpret the HTTP request generated during form submission.

Because the method is POST, the browser sends the field values in the request body. For a student named Asha Kumar the request looks like this:

```
POST /register HTTP/1.1
Host: portal.example.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 37

name=Asha+Kumar&email=asha%40mail.com
```

- **Request line:** the method (POST), the path the server must handle (**`/register`**, taken from the form's action) and the HTTP version.
- **Headers:** **`Host`** names the website, **`Content-Type`** tells the server how to decode the body, and **`Content-Length`** gives the body size in bytes. Browsers also add headers such as User-Agent, Accept and cookies.
- **Blank line:** separates the headers from the body.
- **Body:** the **`name=value`** pairs of the fields, joined by **`&`**. Special characters are percent-encoded (the space becomes **`+`** and **`@`** becomes **`%40`**). Each pair uses the field's **`name`** attribute, which is why inputs must have one.

4. Analyze the HTTP response cycle from server to browser.

- The server receives the request, uses the path and method to pick the handler, and validates the submitted values.
- It saves the registration (for example in a database) and builds a response: a status line, headers and, when needed, a body.
- A common design is Post/Redirect/Get. The server answers **`302 Found`** with **`Location: /thank-you`**, so refreshing the page does not submit the form again.
- The browser follows the redirect with a new GET request, receives **`200 OK`** with the HTML page, parses it into the DOM, loads linked CSS, JavaScript and images, and renders the page.
- If something goes wrong, the status code tells what happened: 400 for invalid input, 404 for a wrong path and 500 for a server error. The 2xx, 3xx, 4xx and 5xx groups mean success, redirection, client error and server error.

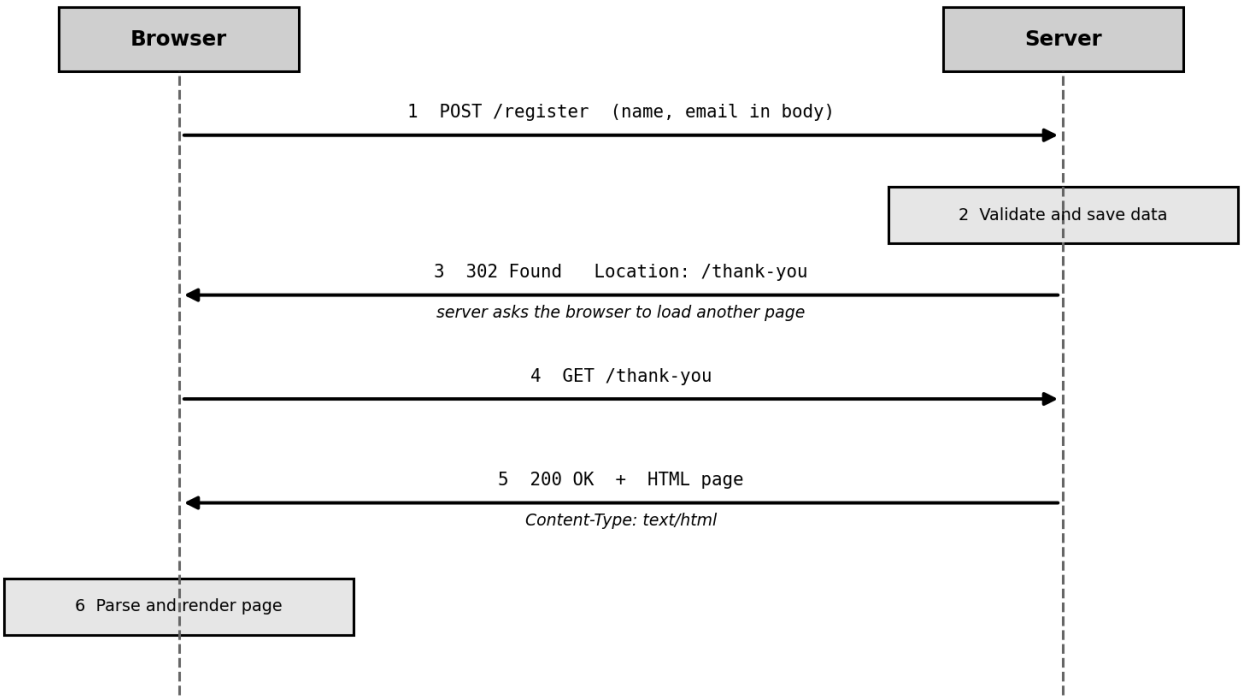


Figure 2: Request and response cycle for the registration form

5. Suggest improvements for usability and accessibility.

- Use visible labels linked with **`for`** and **`id`**. A placeholder alone disappears when typing and is not a replacement for a label.
- Add **`required`**, correct input types (**`email`**, **`tel`**) and **`autocomplete`** so the browser can check and auto-fill data. Show clear, specific error messages next to the field and announce them with **`aria-live`**.
- Give the button a meaningful text such as "Register" and make sure the whole form works with the keyboard, with a visible focus outline and a logical tab order.
- Keep good colour contrast, readable font size and a responsive layout (viewport meta tag) for phones, and set the page language with **`lang`**.
- Show a confirmation page or message after success, and protect the form with HTTPS and a CSRF token.

> **Conclusion:** The portal needs a complete, semantic page with labelled fields. The POST request carries the data in its body, the server answers with a redirect and a page, and the improvements make the form usable by everyone.

Question 2: Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags.

- The **`<h1>`** is never closed (no **`</h1>`**) and the **`<p>`** is never closed (no **`</p>`**).
- Because of this the paragraph is nested inside the heading, and only the closing **`</div>`** ends both of them. A paragraph is not allowed inside a heading, so this nesting is invalid.
- The page also has no DOCTYPE, no **`lang`** on the html tag and no charset declaration.

2. Analyze how different browsers may interpret this structure.

Modern browsers use the HTML5 parsing rules, which say exactly how to repair errors, so they build the same tree: the p ends up inside the h1, and "About us" inherits the large bold heading style (Figure 3). Older browsers, however, had their own error-recovery methods, and they could end the heading earlier, nest the elements differently or apply different margins. This is a common reason for inconsistent layouts. The missing DOCTYPE makes it worse: browsers switch to quirks mode, where box sizing, line height and spacing are calculated in browser-specific ways. Each browser also has its own default style sheet for h1, p and body margins, so without a CSS reset the same markup looks slightly different in each one.

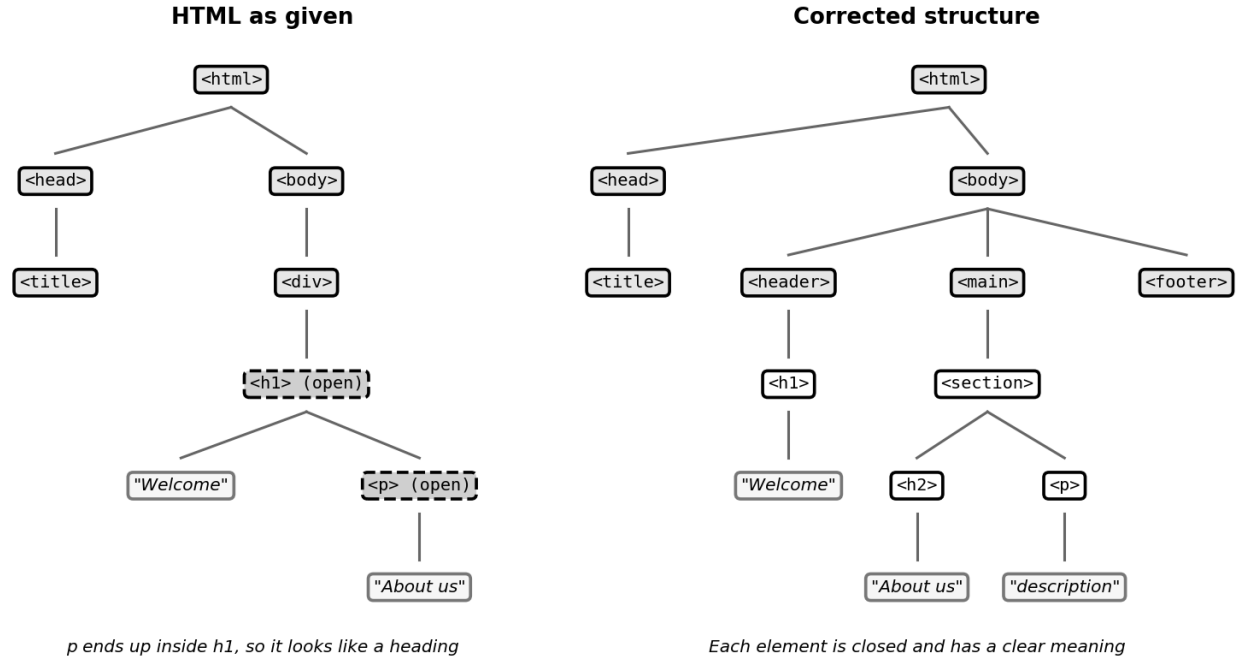


Figure 3: DOM tree built from the given HTML and the corrected tree

3. Identify missing semantic elements (e.g., header, section).

- **`<header>`** for the page heading and introductory content.
- **`<main>`** to mark the main content of the page, and **`<section>`** (with its own heading) for the "About us" content.
- **`<footer>`** for closing information, and **`<nav>`** once the site has a menu.
- A proper heading hierarchy: **`<h1>`** for the page title and **`<h2>`** for the section.

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>
  <main>
    <section>
      <h2>About us</h2>
      <p>Add a short description of the company here.</p>
    </section>
  </main>
  <footer>
    <p>&copy; 2026 Company</p>
  </footer>
</body>
</html>
```

Every element is closed in the right order, the page uses standards mode, and the content is split into meaningful regions. The result can be checked with the W3C Markup Validator.

5. Suggest techniques to ensure cross-browser compatibility.

- Always start with the HTML5 DOCTYPE and validate the markup with the W3C validator, so no browser has to guess.
- Use a CSS reset or normalize.css to remove default style differences, and set **`box-sizing: border-box`**.
- Use modern layout tools (Flexbox and Grid) with fallbacks, and add vendor prefixes automatically with a tool such as Autoprefixer.
- Use feature detection (**`@supports`**, Modernizr) and polyfills instead of checking which browser is running. Check support on caniuse.com before using a new feature.
- Apply progressive enhancement: the basic content and function must work everywhere, and extra features are added on top.
- Use relative units and media queries for responsive design, and test on Chrome, Firefox, Safari and Edge and on real devices or with BrowserStack and DevTools device mode.

> **Conclusion:** The inconsistent layout comes from unclosed tags, wrong nesting and quirks mode. Valid, semantic HTML plus a CSS reset and regular testing gives the same result in every browser.

Question 3: Form Submission Failure

1. Identify issues in the form configuration.

- **`<button type="button">`:** this is a plain button with no submit behaviour. It should be **`type="submit"`** (the default for a button inside a form).
- **`method="GET"`:** a login form must not send a password in the URL. It should use POST.
- **No labels or ids:** the inputs have no **`<label>`**, which hurts accessibility and usability.
- **No validation or autocomplete:** there is no **`required`** and no **`autocomplete`** hint, so empty values can be sent and password managers cannot help.
- **Transport:** nothing shows that the form is served over HTTPS.

2. Analyze why the HTTP request is not triggered.

A form is submitted only when the user activates a submit button (or when script calls **`form.submit()`**). A button with **`type="button"`** has no default action, and there is no JavaScript click handler attached, so clicking "Login" does nothing and no request appears in the Network tab. Pressing Enter does not help either: a form with several text-type fields and no submit button is not submitted by implicit submission. The data therefore never leaves the browser.

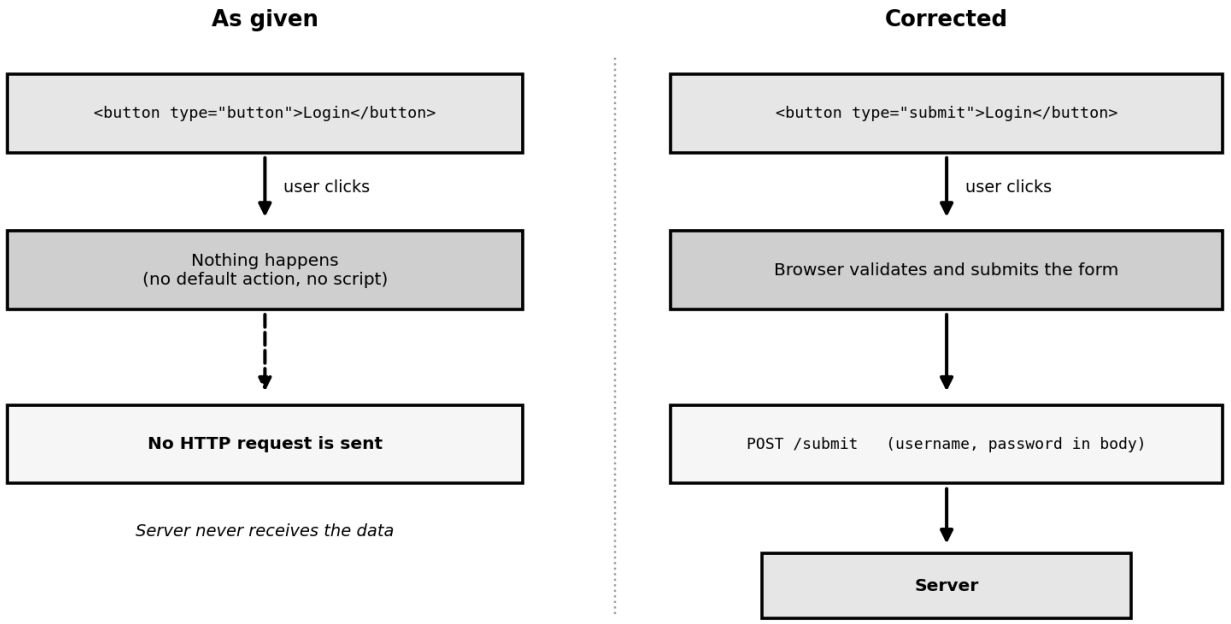


Figure 4: Why the given button sends nothing and how a submit button fixes it

3. Interpret the role of GET method in this scenario.

GET is meant for reading data. If the form were submitted, the browser would put the fields in the query string, for example **`/submit?username=admin&password=1234`**, and send no body. That makes the password visible in the address bar and saved in browser history, bookmarks, proxy logs and server logs. It can also leak through the Referer header and be cached. GET requests are also meant to be safe and repeatable, but a login changes state by creating a session, so GET is the wrong method here.

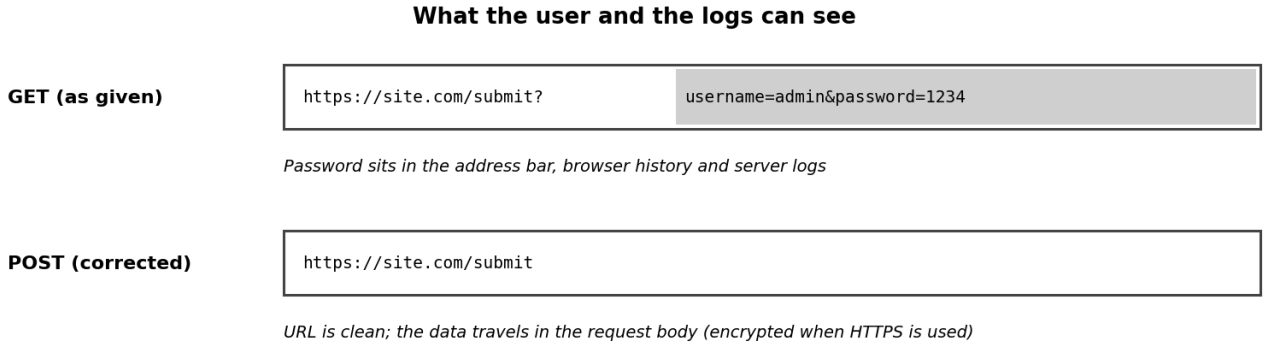


Figure 5: GET exposes the password in the URL, while POST keeps it out of the URL

4. Propose a corrected implementation.

```
<form action="/submit" method="post">
  <p>
    <label for="username">Username</label>
    <input type="text" id="username" name="username"
           required autocomplete="username">
  </p>
  <p>
    <label for="password">Password</label>
    <input type="password" id="password" name="password"
           required autocomplete="current-password">
  </p>
  <button type="submit">Login</button>
</form>
```

The button now submits the form, POST keeps the data in the body, labels are linked to the inputs, and the form is checked before it is sent.

5. Suggest improvements for secure data transmission.

- Serve the whole site over HTTPS (TLS) and enable HSTS, so the data is encrypted in transit. Redirect all HTTP traffic to HTTPS.
- Keep credentials out of URLs and logs. Send **`Cache-Control: no-store`** on login responses.
- Store passwords only as salted hashes (bcrypt, scrypt or Argon2) and compare hashes on the server.
- Add a CSRF token to the form, and set session cookies with the Secure, HttpOnly and SameSite flags.
- Validate input on the server and use prepared statements to prevent SQL injection. Show a generic error such as "invalid username or password".
- Limit failed attempts with rate limiting or lockout, and offer multi-factor authentication.

> **Conclusion:** The form fails because the button never submits it, and it would be unsafe even if it did, because GET puts the password in the URL. A submit button, POST, HTTPS and server-side protection solve both problems.